

Programación 1 — Unidad 4: Trabajo colaborativo

Actividad 3: Ramas, merge y buenas prácticas

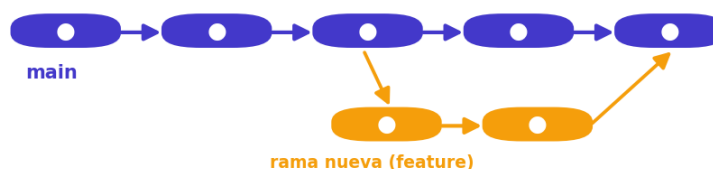
Índice

- 4.6. Ramas (branches)
- 4.7. El archivo .gitignore
- 4.8. Revisión de código entre pares

4.6. Ramas (branches)

Una **rama** es una línea de desarrollo independiente. Te permite trabajar en una nueva funcionalidad sin tocar la versión principal (que por convención se llama `main`), y unirla recién cuando esté lista y probada. Es la base del trabajo en equipo: cada persona avanza en su rama sin molestar a las demás.

Ramas (branches): trabajar en paralelo sin romper la versión principal



Una rama nueva se separa de main, evoluciona en paralelo y luego se fusiona de vuelta.

```
git branch nueva-funcion
git checkout nueva-funcion
git checkout main
git merge nueva-funcion
```

Qué hace el código:

- `git branch nueva-funcion` crea una rama nueva con ese nombre, partiendo del estado actual.
- `git checkout nueva-funcion` te «cambia» a esa rama: a partir de ahí, tus commits van ahí y no afectan a `main`.
- `git checkout main` te devuelve a la rama principal cuando terminaste de trabajar en la

nueva.

- `git merge nueva-funcion` fusiona (trae) los cambios de la rama nueva dentro de `main`.

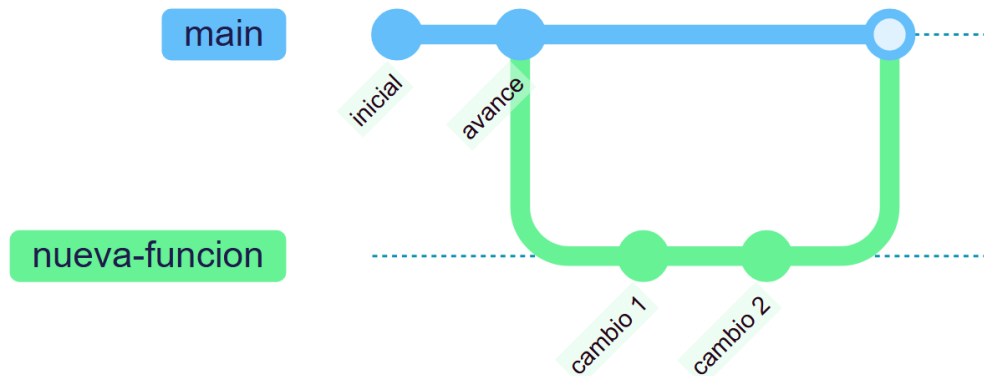


Diagrama 4.6 generado con Mermaid.

4.7. El archivo `.gitignore`

No todo lo que hay en la carpeta del proyecto debe versionarse. Archivos temporales, entornos virtuales o claves privadas no deberían subirse. El archivo especial `.gitignore` contiene una lista de lo que Git debe **ignorar**.

```
# .gitignore para un proyecto Python
venv/
__pycache__/
*.log
.env
```

Qué hace el código:

- La primera línea es un comentario (empieza con `#`): Git también ignora estos comentarios.
- `venv/` le dice a Git que ignore por completo la carpeta del entorno virtual.
- `__pycache__/` ignora la carpeta donde Python guarda archivos temporales.
- `*.log` ignora todos los archivos terminados en `.log` (el `*` significa «cualquier nombre»), y `.env` ignora el archivo de claves.

Buenas prácticas

Creá el `.gitignore` al iniciar el proyecto, antes del primer commit. Subir por error una carpeta `venv` o un archivo con contraseñas es un problema común y a veces difícil de revertir.

Encontrá el error

Acá no hay ningún comando mal escrito. El problema es de flujo de trabajo: pensá qué buena

práctica de esta unidad no se está respetando.

```
git checkout main
git pull
# ... se modifica código delicado directo en main ...
git commit -am "cambios experimentales"
git push
```

Qué estaba mal

El problema es trabajar directamente sobre main con cambios experimentales. Si algo sale mal, se rompe la versión estable para todo el equipo. Lo correcto es crear una rama (git checkout -b nueva-funcion), trabajar ahí, y fusionar a main recién cuando esté probado.

Practicá la lectura

Explicá con tus palabras qué te está informando Git en esta salida, y qué comando usarías a continuación para preparar ese cambio antes de confirmarlo.

```
$ git status
En la rama main
Cambios no rastreados para el commit:
  modificado:   app.py

sin cambios agregados al commit
```

Para reflexionar y autoevaluarte

- ¿Qué comando de Git usás con más confianza y cuál todavía te genera dudas?
- ¿Por qué te parece que trabajar con ramas es una práctica tan recomendada en equipos de desarrollo?

Antes de seguir, verificá que podés:

- Inicializar un repositorio y hacer el primer commit.
- Explicar las tres áreas de Git (directorio de trabajo, staging y repositorio).
- Crear una rama, trabajar en ella y fusionarla con main.
- Explicar para qué sirve un archivo .gitignore.

4.8. Revisión de código entre pares

El código no se escribe solo para que lo lea la computadora: también lo van a leer otras personas — compañeros de equipo, quien mantenga el proyecto en el futuro, o vos mismo dentro de unos meses. Por eso, en el desarrollo profesional es habitual que nadie fusione un cambio a la rama principal sin que otra persona lo revise primero.

A esta práctica se la llama code review (revisión de código), y en plataformas como GitHub se hace a través de un pull request: quien hizo el cambio lo propone, y otra persona lo lee, deja comentarios sobre líneas específicas, y lo aprueba o pide ajustes antes de fusionarlo.

Buenas prácticas

Al revisar el código de otra persona, señalá también lo que está bien hecho, no solo lo que hay que corregir. Y al recibir comentarios sobre tu código, recordá que están evaluando el código, no a vos: es una de las formas más rápidas de aprender.

Nota

Revisar código ajeno también te hace mejor programador o programadora: leer soluciones distintas a las tuyas amplía tu forma de pensar un problema, más allá de si ese código termina fusionándose o no.